

Semester Product Modernization Blueprint

Purpose: A 12-point architecture and roadmap blueprint to make Semester more competitive across student UX, native LMS workflows, backend/data platform, operations console, and SaaS business operations.

Benchmarking rule: Use category-level market expectations unless a named competitor's current public capability has been verified. The goal is not feature parity; it is a differentiated, trustworthy university platform.

1. Strategic product position

```
Semester = student action layer
+ governed learning platform
+ native LMS capability
+ source-aware data platform
+ institution operations layer
+ trusted AI system
+ implementation and service operation
```

Core differentiators to preserve

```
Student clarity over dashboard clutter.
Student control over surveillance.
Source/freshness/authority labels over opaque data ingestion.
Course- and policy-aware AI over generic chat.
Accessible workflows over compliance-only claims.
Integration-first architecture with migration readiness.
Operational evidence and support over demo-only breadth.
```

2. 12-point modernization blueprint

1. Unified AppShell and context-first UX

Gap addressed: Fragmented modules, duplicated navigation, route-specific UI patterns, weak context.

```
Build:
- Five student destinations: Today, My Path, Search, Plan, Me.
- Contextual modules: Study Studio, Ask Semester, Workspace, Career, Campus.
- Shared ContextHeader: title, source/freshness, privacy scope, actions.
- Shared Action Center across every module.
- Global search/command palette.
- Consistent loading/empty/error/offline/no-permission states.
```

Success evidence: every authenticated route passes shell, mobile, keyboard, source/privacy, and visual-regression checks.

2. Today as a decision workspace

Gap addressed: Generic dashboard design and cognitive overload.

Build:

- One Most Important action.
- Three to five next actions.
- Upcoming commitments.
- Path Snapshot and plan health.
- “What changed since your last visit?”
- Explain-before-recommend pattern.
- Snooze/dismiss/correct/share/help controls.
- No “at risk,” “behind,” or surveillance-style labels.

Success evidence: increased first meaningful action completion and student clarity without increased support friction.

3. Decision Packets for high-value moments

Gap addressed: Disconnected tools and unstructured handoffs.

Build:

- Registration Packet.
- Advisor Packet.
- Study Packet.
- Career Packet.
- Study Abroad Packet.

Every packet includes:

context, sources, assumptions, selected options, next actions, human handoff, share scope, export, audit, and expiration/freshness where relevant.

Success evidence: students complete planning/advising workflows and staff receive better-prepared, scoped context.

4. Contextual Ask Semester

Gap addressed: Generic AI chat, ambiguous context, unclear policy boundaries.

Build:

- Context tray: what the assistant can use and cannot use.
- Source tray: course materials, plan, calendar, freshness, exclusions.
- Modes: Explain, Practice, Plan, Draft, Execute.
- Course/assignment/institution policy enforcement.
- Citations, limitations, feedback, and safe alternatives.
- Draft-to-action handoffs with exact preview/confirmation.
- Private, editable, deletable user memory.

Success evidence: grounded response quality, policy enforcement, citation usability, user usefulness, safe fallback rate.

5. Accessible native LMS advantage

Gap addressed: LMS usability debt and assessment/gradebook risk.

Build:

- Accessible Course Studio.
- Content modules, versions, conditional release, student preview.
- Assignment/submission durability, receipts, version history.
- Assessment autosave/resume/accommodations/recovery.
- Rubrics, accessible feedback, batch grading, grade audit history.
- Student grade/what-if view.
- Course outcomes, archive, export.

Success evidence: real course lifecycle passes faculty/student/accessibility/load/recovery UAT.

6. Integration Control Plane

Gap addressed: One-off, opaque connectors.

Build:

- Connector catalog.
- Tenant configuration.
- OAuth/mTLS/LTI identity/security.
- Field mappings and data classifications.
- Source/freshness metadata.
- Webhook verification.
- Durable inbox/retry/dead-letter.
- Reconciliation and discrepancy resolution.
- Circuit breaker/fallback.
- Customer-visible connector health.

Success evidence: every connector is tenant-scoped, observable, reconcilable, documented, and safely disableable.

7. Source-of-truth and data-lineage engine

Gap addressed: Opaque data and untraceable recommendations.

Every material fact stores:

source_kind
source_system
source_record_id
source_url when safe
verified_at/synced_at
authority/confidence
data_classification
retention policy
mapping version

```
access policy
conflict-resolution rule
official fallback
```

Success evidence: source/freshness shown to user where it matters; data issues are traceable and correctable.

8. Policy-as-code and governed configuration

Gap addressed: Manual institution exceptions and brittle settings.

```
Version and test:
- AI policy.
- Assessment policy.
- Feature policy.
- Data-sharing policy.
- Retention policy.
- Notification policy.
- Analytics suppression policy.
- Integration scope policy.
- Support access policy.
- Accessibility policy.
```

Success evidence: changes have owner, approver, scope, effective date, test, audit event, rollback, and customer communication decision.

9. High-assurance tenant and role architecture

Gap addressed: Over-broad access and weak institutional governance.

```
Access decision:
identity + membership + scoped role grant + capability + object rule + consent +
policy + feature entitlement + expiry.
```

```
Build:
- Tenant hierarchy.
- Role/capability matrix.
- ABAC/object authorization.
- Database RLS and storage isolation.
- Time-bound support access.
- Privileged access review.
- Break-glass controls.
- Immutable audit logs.
```

Success evidence: positive/negative/refusal/revocation tests for every critical role and workflow.

10. Reliability for academic critical periods

Gap addressed: SaaS reliability that does not account for registration/finals/grade deadlines.

Build:

- Critical journey SL0s and error budgets.
- Synthetic journey checks.
- RUM: LCP/INP/CLS/errors.
- Assessment/registration/gradebook load tests.
- Academic operations calendar.
- Release freezes.
- On-call/incident commander.
- Backup/restore/DR drills.
- Status/customer communication.
- Game days for SSO/LMS/SIS/AI/database/queue failure.

Success evidence: drills, restore reports, load reports, peak-period signoff, incident corrective actions.

11. Institution Launch Method and customer operating platform

Gap addressed: Custom, opaque, slow implementation.

Semester Launch Method:

Discover → Design → Configure → Connect → Validate → Train → Launch
→ Hypercare → Measure → Expand.

Build:

- Institution Launch Center.
- Implementation projects/RACI/risk/decision log.
- UAT scripts and go-live gates.
- Customer portal.
- Training academy.
- Customer success/QBR/renewal workflows.
- Data portability/offboarding.

Success evidence: repeatable launch from signed contract to hypercare across multiple customers.

12. Student portability and evidence graph

Gap addressed: Student work/skills trapped in disconnected systems.

Build:

- Student Credential Wallet.
- Skills/evidence graph.
- Student-selected artifacts.
- Scoped verification requests.
- Institution-issued credential record.
- Private-by-default sharing.
- Time-limited links, revocation, view history.

- Export and portable record roadmap.
- CASE/Open Badges/CLR interoperability phases.

Success evidence: student can add evidence, request scoped verification, share/revoke, and export without exposing restricted data.

3. Front-end prioritization

3.1 Front-end priority order

P0 — launch-blocking experience foundations

1. Unified AppShell and navigation.
2. Real login/signup/auth-state UX.
3. Consistent source/freshness/privacy components.
4. Design tokens/component library.
5. Loading/empty/error/offline/no-permission states.
6. Keyboard, focus, semantic, contrast, reflow baseline.
7. Mobile-first core routes.
8. Save/autosave/recovery feedback.

P1 — retention-driving student workflows

9. Today/Action Center decision workspace.
10. My Path and Registration Center.
11. Plan/schedule conflict/backup course workflow.
12. Advisor Packet and safe sharing.
13. Contextual Ask Semester.
14. Study Studio/source workflow.
15. Search with source-aware results.

P2 — faculty/institution adoption

16. Course Studio.
17. Assignment/submission/recovery.
18. Assessment and accommodations.
19. Gradebook and feedback.
20. Institution admin and policy UI.
21. Integration-health and source governance UI.
22. Implementation/UAT/training UI.

P3 — differentiation and ecosystem

- 23. Career/evidence graph.
- 24. Credential Wallet.
- 25. Campus Hub and opportunity marketplace.
- 26. Study abroad/athlete/transfer modules.
- 27. Developer portal and integration sandbox.

3.2 Front-end scoring gate

A feature cannot leave preview without:

- [] Shared component use and token compliance.
- [] 320px mobile, tablet, desktop behavior.
- [] Keyboard-only task success.
- [] Screen-reader/basic semantic test.
- [] Visible focus and contrast check.
- [] Loading, empty, error, offline, stale, permission states.
- [] Source/freshness/privacy/estimate treatment where applicable.
- [] Analytics and error monitoring.
- [] Feature flag and rollback.
- [] Support/help route.

4. Backend architecture choices

4.1 Architectural style

Start with a modular monolith with clear domain boundaries, not premature microservices.

Why:

- Faster consistency across product/LMS/integration/security domains.
- Easier tenant-aware transactions and auditability.
- Lower operational complexity while the model evolves.
- Clear extraction path for high-scale domains later.

4.2 Domain boundaries

Identity and authorization
Tenant/customer configuration
Student planning/action graph
Course and LMS
Assessment and gradebook
Workspace/files/materials
Search/retrieval
AI intelligence
Integrations and synchronization
Notifications
Billing/entitlements

Support/customer success
Compliance/audit
Observability/reliability

4.3 Data architecture

Transactional store:
Postgres/Supabase with RLS for tenant/user/object isolation.

Event/audit store:
Append-only audit/event tables; optional external immutable archive.

Search index:
Permission-aware index with tenant/object/source access filter.

Object storage:
Scoped storage paths, signed access, malware/type validation, lifecycle policy.

Queue/job layer:
Durable jobs, idempotency, retry policy, dead-letter queue, reconciliation.

Analytics:
Separate governed aggregate analytics model; no direct ad hoc production record browsing.

Evidence/compliance store:
Versioned, controlled documents/evidence with expiration/review metadata.

4.4 Integration architecture

Institution gateway / provider

- secure connector adapter
- schema/signature/timestamp/replay validation
- durable inbox/idempotency/order metadata
- source record and field mapping
- normalized read model
- source/freshness metadata
- tenant-scoped product API
- reconciliation/discrepancy queue
- customer-visible health and fallback

4.5 Security architecture

Server-side authorization for sensitive actions.
Database RLS for every tenant/user scoped record.
Storage access policy and signed URLs.
MFA and SSO for privileged users.
Secret manager; never client-side service keys.
Scoped OAuth/mTLS/LTI credentials.
Audit events for sensitive actions.

Feature flags and kill switches.
Threat models and automated negative tests.

4.6 AI architecture

User request
→ tenant/course/assignment context
→ policy evaluation
→ source-access validation
→ authorized retrieval
→ prompt-injection-resistant context assembly
→ provider/model route
→ output safety/policy/grounding evaluation
→ citations/limitations/actions
→ confirmation for writes
→ audit/telemetry/evaluation feedback

5. Higher-ed operations console structure

5.1 Console principles

Read-first.
Least privilege.
No generic student-record browsing.
Tenant/customer context always visible.
Every high-risk write needs reason, confirmation, audit, and where appropriate approval.
Operations designed around actual work queues, not database tables.

5.2 Core modules

Module	Primary job	Main users
Command Center	Prioritize urgent work/risk	Executives, operations leads
Customers	Customer 360, entitlements, contacts, success, renewal	CSM, sales ops, implementation
Tenants	Configure lifecycle, policy, modules, feature flags	Implementation, platform admins
Identity/Access	Roles, grants, reviews, support access, audit	Security, platform admins
Implementation	Project, UAT, training, go-live, hypercare	Implementation managers
Support	Tickets, incidents, communications, knowledge	Support/SRE/CS
Integrations	Connectors, freshness, mappings, reconciliation	Integration engineers, customer IT
Reliability	SLOs, error budgets, backups, DR, critical periods	SRE, leadership
Compliance	HECVAT/SOC2/VPAT/privacy/AI/vendors/evidence	Security/privacy/accessibility/legal
Revenue	Quotes, contracts, subscriptions, invoices, dunning	Finance, commercial ops
Product Quality	Feedback, a11y, UX friction, AI quality, releases	Product/design/engineering

Module	Primary job	Main users
Audit	Search immutable sensitive actions	Security/legal/audit

5.3 Academic operations modes

Registration mode:

Registration source health, seat/availability freshness, release freeze, integration alerts, support queue.

Assessment mode:

Autosave/submission health, accommodations, assessment load, P0 coverage, status communications.

Grade mode:

Gradebook latency, posting state, passback/reconciliation, faculty support.

Term-start mode:

SSO, roster sync, course publish, content import, onboarding, support volume.

5.4 Role tiers

Tier 0: Executive read-only insights.

Tier 1: Customer success, implementation, support, finance, sales operators.

Tier 2: Integration, SRE, security, privacy, accessibility, AI specialists.

Tier 3: Controlled privileged administrators.

Tier 4: Time-limited, audited, two-person break-glass access.

5.5 Console UX requirements

Action queue first.

Environment/tenant context always visible.

Accessible tables/charts/filters/exports.

No color-only severity.

Saved views and keyboard command palette.

Confirmation with impact preview.

Runbook/help at point of action.

Audit link from every sensitive action.

Minimal incident mode for P0/P1 response.

6. Modernization roadmap

Wave 0: Assess and stabilize

Repository/deployment/data/auth audit.

Claims register and capability matrix.

Route/component consolidation.

Demo/production separation.
P0 security/accessibility/reliability remediation.

Wave 1: Product foundation

AppShell/design system.
Real auth/tenant/RLS.
Today/Action Center.
My Path/registration/plan core.
Source/freshness/privacy states.
Observability/backup/rollback.

Wave 2: Institutional trust and workflow

SSO/SCIM.
Integration Control Plane.
Trust/HECVAT/VPAT/DPA package.
Institution Launch Center.
Support/implementation/customer portal.
Policy-as-code.

Wave 3: Native LMS and governed AI

Course Studio.
Assignments/submissions.
Assessment/accommodations.
Gradebook/audit/passback.
LTI/OneRoster/QTII/Common Cartridge.
Contextual Ask Semester and Study Studio.
AI evaluations/kill switch.
Migration/parallel run.

Wave 4: Defensible ecosystem

Credential Wallet.
Skills/evidence graph.
CASE/Open Badges/CLR roadmap.
Career/campus/opportunity network.
Developer portal/sandbox.
Benchmark/research program.

7. Priority decision rule

Prioritize work that does at least two of the following:

- Makes a recurring student decision clearer.
- Reduces enterprise trust/procurement risk.
- Unblocks multiple product modules.
- Prevents loss of student work or data.
- Improves accessibility in a critical workflow.
- Makes implementation repeatable.
- Supports academic peak reliability.
- Creates portable student value.
- Reduces operational/support burden.
- Creates evidence for a defensible market claim.

Defer work that is mainly decorative, duplicates another system without advantage, requires surveillance, cannot be supported, or lacks a clear source/authority/privacy model.